



Course 9

Common Tables

Agenda



- What is Common Table
- Common Table Types
- Common Table Properties
- Common tables behind the scenes
- Built-in tables in each Common Scheme
- `common_local_trx`
- Fabric commands for Common table using population
- CommonDB Backup
- IIDFinder
- Vacuum & maintenance
- Delete Common Table
- Additional Configurations – `config.ini`
- Design Considerations
- Performance Considerations
- Best Practices
- Helpful Kafka commands

What is a Common Table?

The **Common Area** (or **CommonDB**) is a component of Fabric that stores tables and data outside of Logical Unit Instances (LUIs).

Typical Uses of Common Tables:

- Reference Tables
- Lookup Data
- Cross-LUI/LUT Data
- Reports
- Data for downstream applications

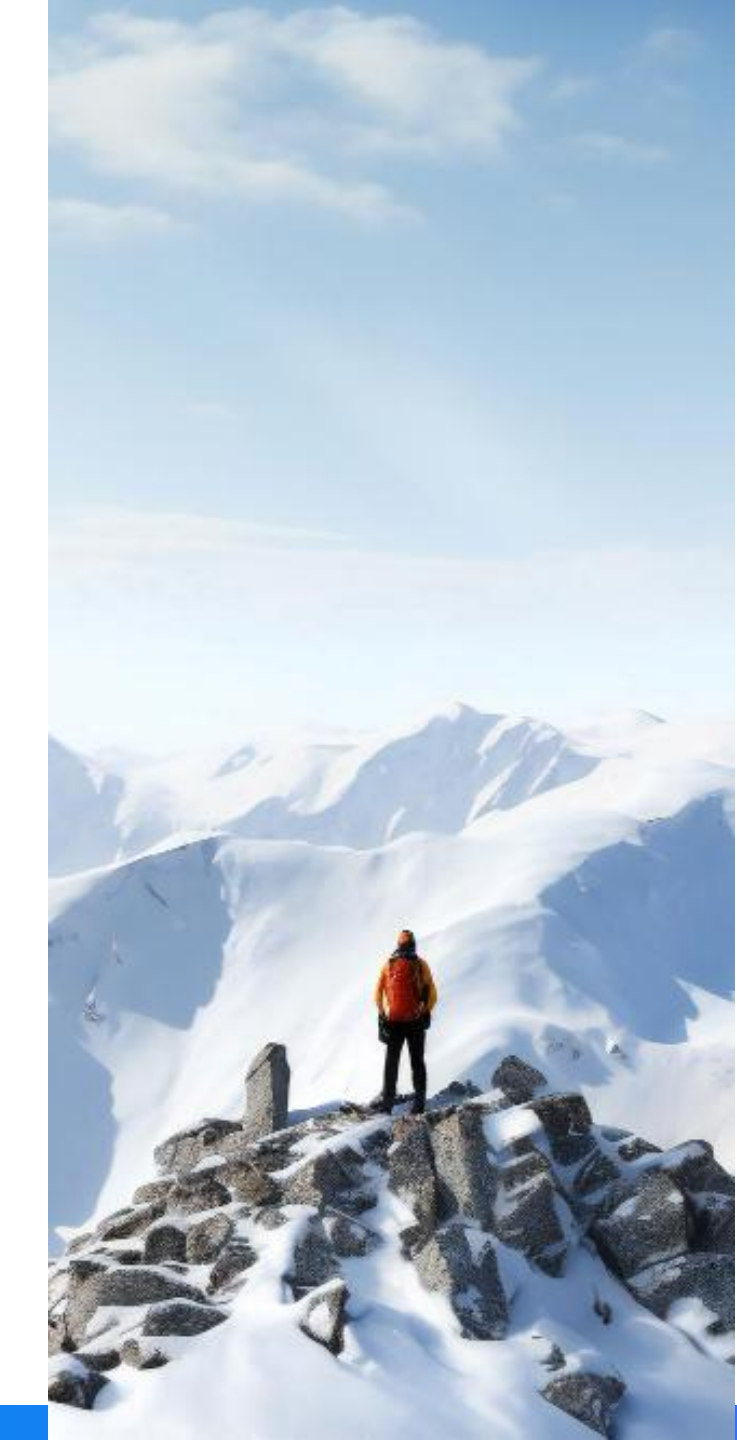
Common tables are stored in dedicated SQLite files within the "**Common Area**", a physical location on the server defined in the config.ini file using the [fabricdb].MDB_COMMONS_PATH parameter.

Default path:

`${FABRIC_HOME}/storage/common`

```
rm -rf /tmp/fabric-deployment-848b7d9f89-6qwqc:~/workspace/storage/common$ ll
total 3492
drwxrwsr-x 2 fabric fabric 4096 Oct 11 17:13 ./
drwxrwsr-x 4 fabric fabric 4096 Jun 26 08:49 ../
-rw-rw-r-- 1 fabric fabric 1093632 Aug 31 14:57 common.db
-rw-rw-r-- 1 fabric fabric 32768 Oct 11 17:13 common.db-shm
-rw-rw-r-- 1 fabric fabric 0 Oct 11 17:13 common.db-wal
-rw-rw-r-- 1 fabric fabric 237568 Oct 11 16:05 mtables.db
-rw-rw-r-- 1 fabric fabric 32768 Oct 11 17:13 mtables.db-shm
-rw-rw-r-- 1 fabric fabric 0 Oct 11 17:13 mtables.db-wal
-rw-rw-r-- 1 fabric fabric 2138112 Oct 11 17:13 ref_crm.db
-rw-rw-r-- 1 fabric fabric 32768 Oct 11 17:13 ref_crm.db-shm
-rw-rw-r-- 1 fabric fabric 0 Oct 11 17:13 ref_crm.db-wal
```





What is a Common Table?

Each node in the Fabric Cluster holds a local copy of the Common tables, enabling SQL JOIN operations between Common tables and LUIs using standard SQL queries.

Fabric ensures these local copies remain synchronized across nodes, keeping all SQLite files up-to-date and available for queries within any Fabric session.

The Common Area may contain one or more SQLite files, with each file representing a different **schema**. The schema for a Common table is determined by the table's "**Schema**" **property**:

1. **If the schema does not exist:** A new SQLite file will be created to store the table.
2. **If the schema already exists:** The table will be added to the corresponding SQLite file.
3. Default when left empty – "common" schema

Common Table Types

There are two types of Common tables:

1. Population-Based Common Tables

These tables rely on a population process to fetch and insert data.

If the **Sync Method** is set to **Time Interval** or **Decision Function**, a COMMONAREA_TABLE_SYNC job is created when the table is deployed.

The job wakes up at intervals defined by the BG_REF_SYNC_INTERVAL_SEC (config.ini) parameter (default: 5 minutes) and triggers the population process according to the table's Sync Method.



Note:

When the Sync Method set to None, job will be created but will not run. REF_SYNC command will trigger the job to be executed

Common Table Types

2. SOR (System of Records) Common Tables

These tables do not define populations. Instead, they are populated through Fabric's internal logic using Fabric transactions.

Example:

```
Db ci = db("fabric");
ci.beginTransaction();
String SQL = "INSERT INTO common_table (field) VALUES (?)";
Object[] params = new Object[]{value};
ci.execute(SQL, params);
ci.execute("commit");
```



Note:

- You can use **Fabric DB Actors** to perform DML operations on Common tables. The Actors must operate within a transaction, either by using transaction stages or by manually managing the transaction with begin, commit, and rollback commands.
- When a Common table is updated within the GET functionality, there is no need to explicitly manage a transaction, as the entire GET operation runs within a transaction. The commit or rollback will be handled automatically based on the success or failure of the GET

Common Table Properties

- **Schema**

The name of the SQLite DB in which this table will be stored.

If left empty, the table will be added to the generic "common" schema (common.db file).



Schema best practice:

Since Common tables are stored in SQLite, any open transaction blocks other users from writing to the SQLite file (reading is possible as Common tables are using WAL file).

Therefore, it is advisable to create separate schemas for tables that:

1. Receive many transactions (not to block other tables)
2. Need to be updated at the same time as other tables. Only sequential update can be done in the same schema



Note: In case of a corrupted DB file, isolating tables in separate schemas minimizes the impact on other tables.

Common Table Properties

- **Index Post Sync**

This setting determines whether an index should be created on a Reference table only after the data is synced.

In such cases, during the sync process:

- A temporary table is created in the Common DB (with the PK only)
- The newly fetched data is loaded into the temporary table.
- Indexes are created on the temporary table.
- The main table is deleted, and the temporary table is renamed to replace it.



Schema best practice:

Index Post Sync is particularly useful for Common tables with population, where the **Truncate Before Sync** property is set to true and the table contains a large volume of data (over 200 million records).

Common Table Properties

- **Required Reference Tables**

When there are dependencies between Common tables (e.g., table **t2** depends on **t1**), the Required Reference Tables property ensures proper synchronization:

- On the first sync of **t2**, it will not proceed unless **t1** has already completed its initial sync.
- Additionally, if a ref_sync is triggered for both tables, they will run sequentially rather than simultaneously, ensuring that dependent tables sync in the correct order.

- **Columns Collation**

See course 1

- **Full-Text Search (FTS)**

See course 1

- **Sync Wait Timeout**

See course 1

- **Sync Method**

See course 1

- **Truncate Before Sync**

See course 1

Example: balance table requires payment table:

```
fabric>ref_sync tables='ref_crm.balance,ref_crm.payment';
|Table name      |
+-----+
|ref_crm.balance|
|ref_crm.payment|

(2 rows)
fabric>ref_status;
|table name      |status      |node
+-----+-----+-----+
|current session transaction|sync error|notes|
+-----+-----+-----+
|ref_sor.sor      |          |dev-fabric-de
|common.contract_offer_mapping|READY      |dev-fabric-de
|ref_crm.balance   |WAITING_FOR_SYNC|dev-fabric-de
|ref_crm.offer     |READY        |dev-fabric-de
|ref_crm.payment   |IN_SYNC      |dev-fabric-de
ess|          |

(5 rows)
```

Attach Common tables to LUT

The References tab displays a list of Common Reference tables defined in the project. A Reference table should be selected only if:

- You want to use the Reference table as a lookup function in one of the populations.
- You need to ensure that GET operations do not run before the Common table completes its first sync.

Note:

- A Reference table can still be accessed through code (e.g., in functions) even if it is not selected in the References tab.
- The sync status of a Common table is verified against the k2_objects_info table within the Common table's schema. If a record exists, the GET operation can proceed.
- It is recommended to limit the number of selected Reference tables to prevent unnecessary delays, as each GET operation will validate the sync status of all selected Common tables.
- This configuration also affects GET operations within sessions that manually invoke the REF_SYNC command. In such cases, the GET will wait for all selected Reference tables to complete their sync before proceeding, even if the Common tables were already synced earlier.

Common tables behind the scenes

The **CommonDB** consists of an SQLite file per schema (as defined in the Common table's "Schema" properties) that holds all the reference tables.

1. Session Initialization:

- When a session is opened, Fabric automatically attaches all the SQLite files of the CommonDB to the session, enabling join queries to be executed.

2. Distributed Environment and Synchronization:

- Since Fabric is a distributed environment, each Fabric node contains all the CommonDB files.
- Fabric uses an internal Kafka system to synchronize the Common tables across Fabric nodes.
- Every transaction applied to a Common table is not directly applied to the local file. Instead, it is sent to a central Kafka broker to be consumed by all Fabric nodes. Each node then applies the transaction to its local Common tables, including the node that initiated it.

Common tables behind the scenes

Common API logic

The **Common API** acts as the manager for Common tables, residing on each node. It is responsible for both sending updates to all nodes and consuming updates received from other nodes.

To support this functionality, Fabric creates the following for each Common table once it's deployed:

1. **Kafka Topic** to send and receive transactions for the table.
 - **Topic Name:** common_area_<SCHEMANAME>.<TABLENAME>_<SPACENAME>_<CLUSTER_ID>
2. **Staging Area for Long Transactions** (explained later)
 - **Database Staging:** Keyspace K2_ref/k2_commons with a table named "snapshots".
 - **Kafka Staging:** Topic named common_area_internal_<SCHEMANAME>.<TABLENAME>_<SPACENAME>_<CLUSTER_ID>.
3. **On Each Node:**
 - **Table Creation:** The Common table is stored in the SQLite file based on its Schema property.
 - **Kafka Producer:** Each node has a Kafka producer to send messages for that table from sessions running on that node.
 - **Kafka Consumer:** Consumes messages for that table and apply them to the local Common DBs.
4. **Population-Defined Tables:**
 - If the **Sync Method** is **Time Interval** or **Decision Function**, a COMMONAREA_TABLE_SYNC job is created.
 - This job is responsible for executing the population every BG_REF_SYNC_INTERVAL_SEC minutes (it will run or not depending on its Sync Method).

Common tables behind the scenes

Common API logic: Outgoing Transactions - Producer Flow

All DML commands executed on Common tables between the "begin" and "commit" or "rollback" statements define a single "transaction" (the transaction "scope"). Each transaction is handled separately by the Common API.

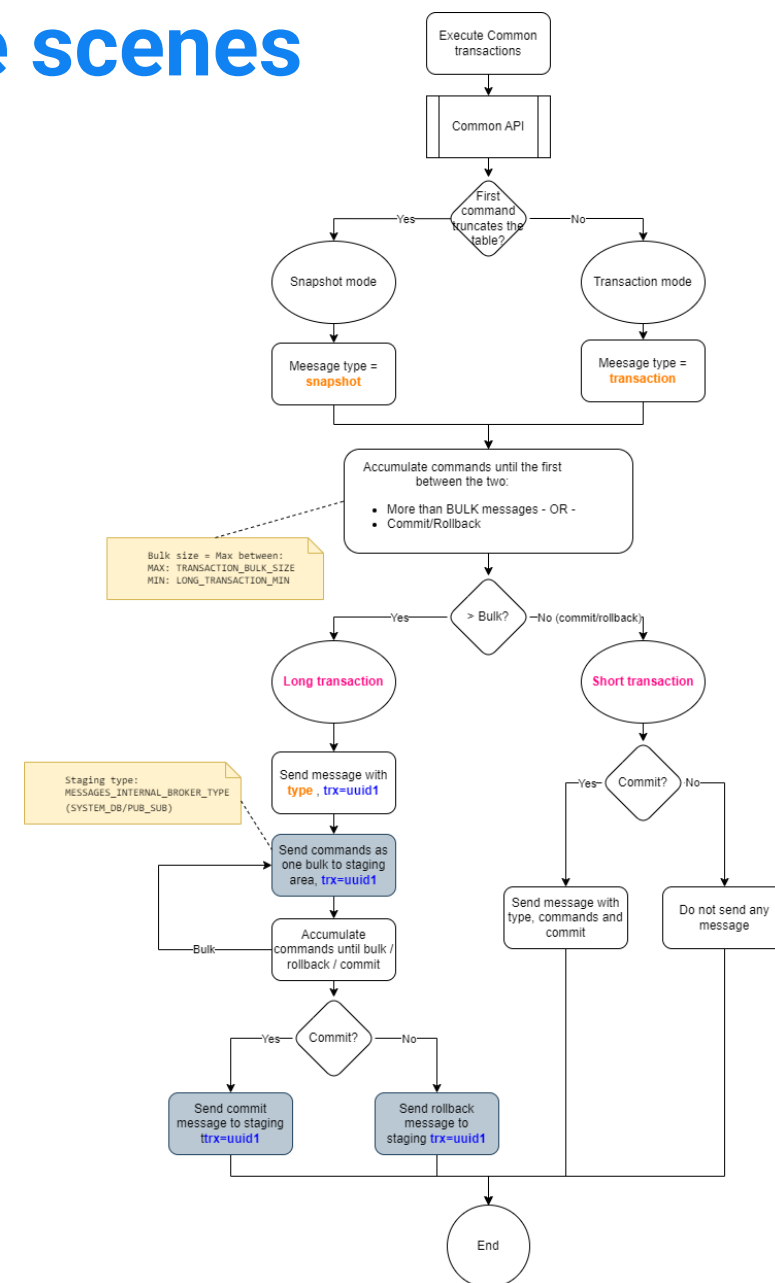
The Common API analyzes each transaction based on two main parameters, which determine how the transaction will be processed:

1. Transaction Mode:

If the first command in the transaction deletes all data in the table - either by issuing a TRUNCATE command or a DELETE * without a WHERE clause - the transaction is considered to be in "**SNAPSHOT**" mode. If not, the transaction is in "**TRANSACTION**" mode.

2. Transaction Size:

The producer accumulates the commands sent in the transaction. If more than a certain number (TRANSACTION_BULK_SIZE) of commands are executed, it is considered a "**Long**" transaction; otherwise, it is a "**Short**" transaction.



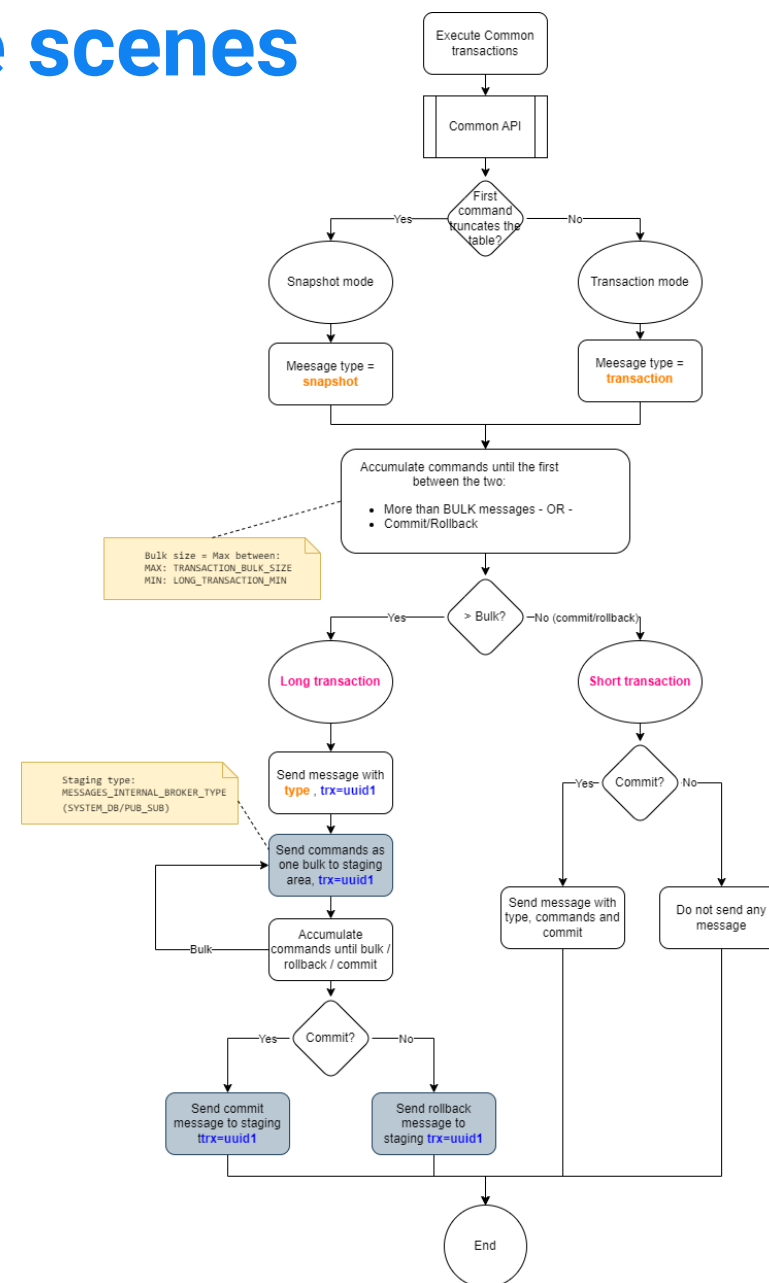
Common tables behind the scenes

Common API logic: Outgoing Transactions - Producer Flow

Producer Error Handling

In the event of an error or session timeout (and commit or rollback were not executed):

- Short Transactions: The message will not be sent to Kafka.
- Long Transactions: If bulk messages have already been sent, a rollback message will be issued.



Common tables behind the scenes

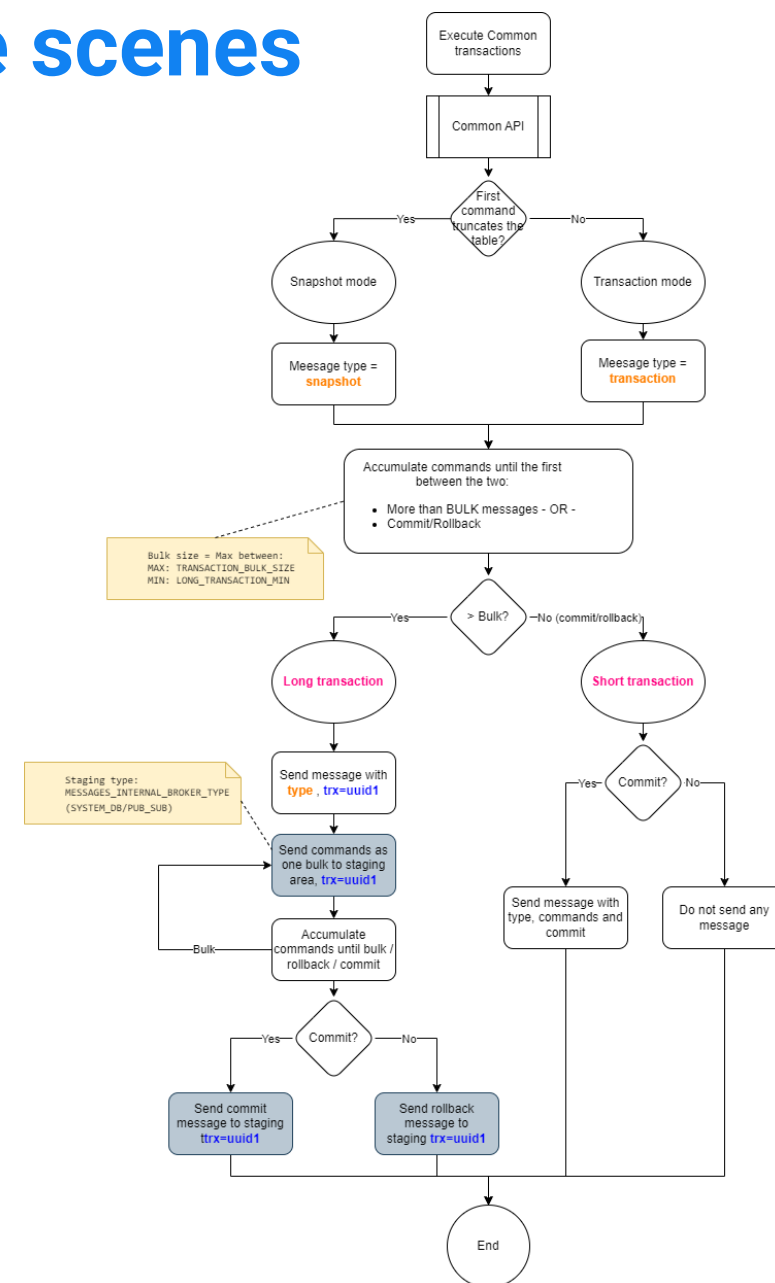
Common API logic: Outgoing Transactions - Producer Flow

Configuration Settings

- 1. Staging Area Configuration:** Set in config.ini under [common_area_config].MESSAGES_INTERNAL_BROKER_TYPE:
 - PUB_SUB: Uses an additional Kafka topic named common_area_internal_<SCHEMANAME>.<TABLENAME>_<SPACENAME>_<CLUSTER_ID>.
 - SYSTEMDB: Uses a table in the system database named k2ref.snapshots or k2common.snapshots.
- 2. Bulk Size:** determined as the maximum between:
 - Maximum Bulk Size: TRANSACTION_BULK_SIZE = 10,000.
 - Minimum Bulk Size: LONG_TRANSACTION_MIN (a hidden parameter).

Purpose of Min and Max Parameters:

- If the number of commands in a transaction is less than TRANSACTION_BULK_SIZE but more than LONG_TRANSACTION_MIN, it is still considered a "Long" transaction.
- This configuration ensures efficient transaction processing by balancing the need for fewer bulks in large tables while still applying the long transaction mechanism for smaller tables.



Common tables behind the scenes

Common API logic: Kafka messages example

	SNAPSHOT		TRANSACTION	
	LONG (Truncate, # of messageas >= bulk size)	SHORT (Truncate, # of messageas < bulk size)	LONG (No truncate, # of messageas >= bulk size)	SHORT (No truncate, # of messageas < bulk size)
Staging	com.k2view.fabric.commonArea.common.BulkMessage BaseMessage java.util.LinkedList @delete from ref_sor.sor INSERT INTO ref_sor.sor (id,fname,lname) VALUES (?, ?, ?) ON CONFLICT(id) DO UPDATE SET fname=?, lname=? @28bdd2c7-29eb-4092-b821-5757a1454b3f com.k2view.fabric.commonArea.common.BulkMessage BaseMessage java.util.LinkedList INSERT INTO ref_sor.sor (id,fname,lname) VALUES (?, ?, ?) ON CONFLICT(id) DO UPDATE SET fname=?, lname=? @28bdd2c7-29eb-4092-b821-5757a1454b3f com.k2view.fabric.commonArea.common.BulkMessage BaseMessage java.util.LinkedList INSERT INTO ref_sor.sor (id,fname,lname) VALUES (?, ?, ?) ON CONFLICT(id) DO UPDATE SET fname=?, lname=? @28bdd2c7-29eb-4092-b821-5757a1454b3f com.k2view.fabric.commonArea.common.SuccessMessage BaseMessage @ 8bdd2c7-29eb-4092-b821-5757a1454b3f		com.k2view.fabric.commonArea.common.BulkMessage BaseMessage java.util.LinkedList INSERT INTO ref_sor.sor (id,fname,lname) VALUES (?, ?, ?) ON CONFLICT(id) DO UPDATE SET fname=?, lname=? @de4e1437-cae8-4ec5-90ac-5ec977e783cc com.k2view.fabric.commonArea.common.BulkMessage BaseMessage java.util.LinkedList INSERT INTO ref_sor.sor (id,fname,lname) VALUES (?, ?, ?) ON CONFLICT(id) DO UPDATE SET fname=?, lname=? @de4e1437-cae8-4ec5-90ac-5ec977e783cc com.k2view.fabric.commonArea.common.BulkMessage BaseMessage java.util.LinkedList INSERT INTO ref_sor.sor (id,fname,lname) VALUES (?, ?, ?) ON CONFLICT(id) DO UPDATE SET fname=?, lname=? @de4e1437-cae8-4ec5-90ac-5ec977e783cc com.k2view.fabric.commonArea.common.SuccessMessage @de4e1437-cae8-4ec5-90ac-5ec977e783cc	
Table's topic	com.k2view.fabric.commonArea.common.SnapRequest startOffset @28bdd2c7-29eb-4092-b821-5757a1454b3f ref_sor.sor	com.k2view.fabric.commonArea.common.BulkMessage BaseMessage java.util.LinkedList @SnapRequest @e68ffb8-daeb-429e-9f36-572682152250t @delete from ref_sor.sor INSERT INTO ref_sor.sor (id,fname,lname) VALUES (?, ?, ?) ON CONFLICT(id) DO UPDATE SET fname=?, lname=? com.k2view.fabric.commonArea.common.SuccessMessage	com.k2view.fabric.commonArea.TransactionRequest startOffset @de4e1437-cae8-4ec5-90ac-5ec977e783cc	com.k2view.fabric.commonArea.common.BulkMessage @BaseMessage java.util.LinkedList @TransactionRequest @6a3ead9-59fe-4d2a-a128-eb4ef28af8d7 INSERT INTO ref_sor.sor (id,fname,lname) VALUES (?, ?, ?) ON CONFLICT(id) DO UPDATE SET fname=?, lname=? com.k2view.fabric.commonArea.common.SuccessMessage



Note:

- **Short Transactions:**
 - The commit wording exists but is not used.
 - If a rollback occurs, the message will not be sent to the topic, so rollback functionality is unnecessary for short transactions.
- **Handling Session Timeout:**
 - **If a bulk was created and sent (long transaction):** A rollback message will be sent when the session times out.
 - **If no bulk was created (short transaction):** The message will not be sent to Kafka, so no rollback message is required.

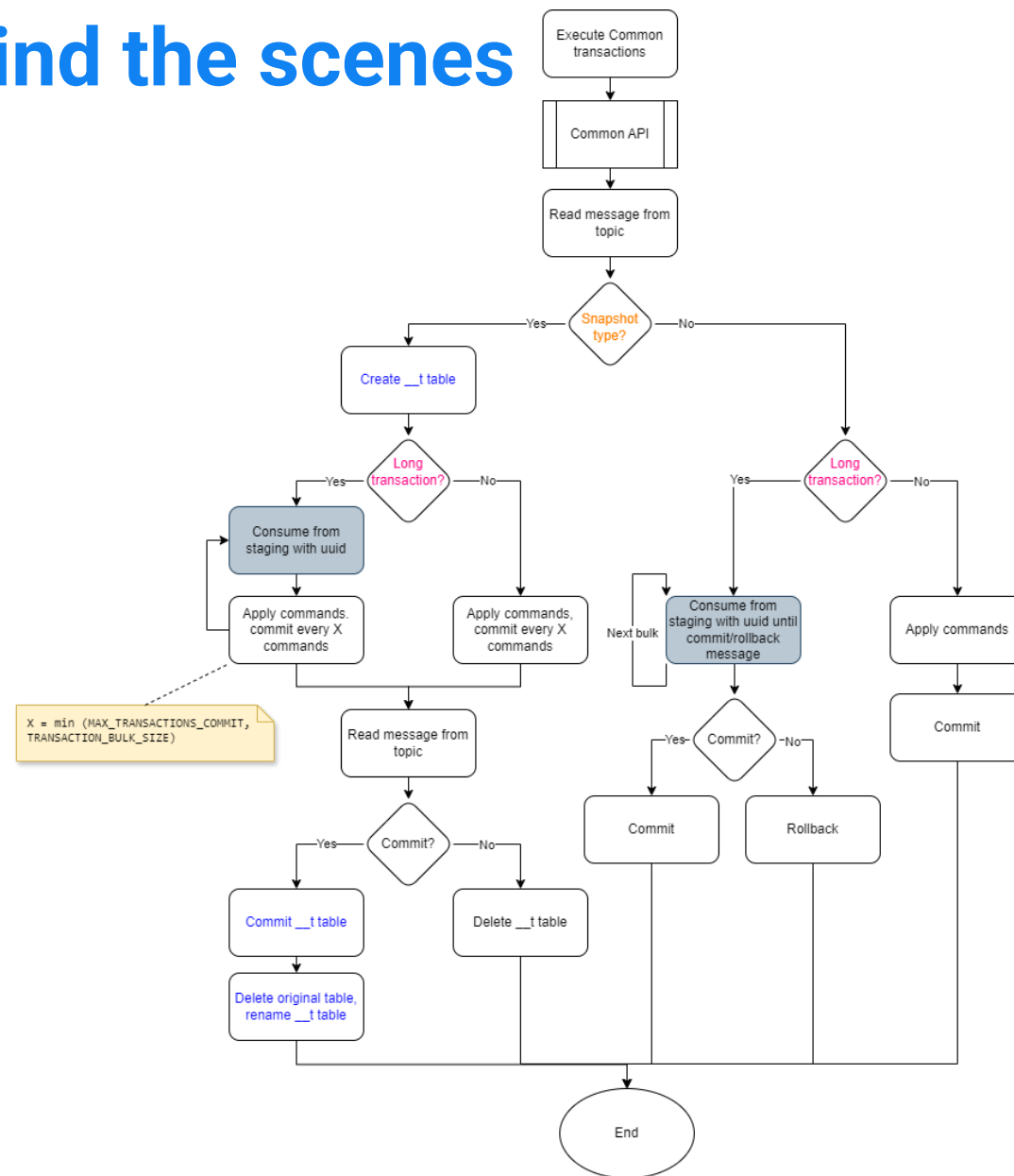
Common tables behind the scenes

Common API logic: Incoming Transactions - Consumer Flow

A Kafka Consumer is automatically created for each Common table on every node.

SNAPSHOT mode messages handling:

1. Create a temporary table named `__t_originalTableName`.
2. Apply all transactions to this temporary table.
3. Execute a commit on the SQLite file after every `MAX_TRANSACTIONS_COMMIT` commands and after each bulk. (The default value of `MAX_TRANSACTIONS_COMMIT` is 100.)
4. Once all transactions are applied, delete the original table and rename the temporary table to the original table's name.



Common tables behind the scenes

Consumer Error Handling

- The consumer reads up to [common_area_pubsub].MAX_POLL_RECORDS messages from Kafka in each poll (default: 500) for performance optimization.
- Kafka commits are performed after processing each batch of MAX_POLL_RECORDS messages, as well as **before and after** each long transaction message.
- For any Kafka-related issues (e.g., connection, commit, or read failures), Fabric will **retry indefinitely** (hard-coded, with no configurable parameters).
- If an error occurs within a message, the consumer retries all messages from the last commit. Each message is then committed individually to Kafka.

Note: This process will significantly reduce performance.

Handling Specific Errors:

- Logical Errors: If the error is logical (e.g., duplicate primary key), the message is logged and skipped without retries.
- Physical Errors:
 - *SQLite Errors (e.g., SQLITE_BUSY, SQLITE_LOCKED):*
Retries are performed every OPERATION_RETRIES_WAIT_MS milliseconds (default: 1000 ms) until OPERATION_RETRIES_COUNT is reached (default: -1, meaning unlimited retries).
- Kafka Commit Errors: Exit and restart processing from the next 500 messages.

Common tables behind the scenes

Consumer Error Handling

- *Long Message Errors (Next Bulk Not Available):*
The consumer waits for the next bulk, checking every `DEFAULT_ACTION_MILLI` milliseconds (default: 100 ms).
Max Wait Time:
 - PUB_SUB: `CONSUMER_IDLE_TIME` = 60000 (ms)
 - Cassandra: `CASSANDRA_WAIT_MESSAGE_TIMEOUT` (ms)
- If the Timeout is Reached:
 - SNAPSHOT Mode: Delete the `__t` table.
 - TRANSACTION Mode: Roll back the SQLite changes.
 - Log the error and skip the message.

Note: if a command is resulting in 0 records affected (like update when the PK is not there, or delete of records that doesn't exists) – No logs will be written, (for performance reasons)

Common tables behind the scenes

Common API logic: Background Sync Flow

If a Common table has a defined population, a COMMONAREA_TABLE_SYNC job is created to manage ongoing population.

- **Job UID:** COMMONAREA_TABLE_SYNC_<SCHEMANAME>.<TABLENAME>
- **Job Activation:** The job wakes up every BG_REF_SYNC_INTERVAL_SEC seconds (default is 5 minutes) and activates the population process according to its Sync Method.
*The job is created also if Sync Method is set to None, but will not be activated.

Once the population is activated, the fetched records are sent **as one transaction** to the Kafka topic, where they are consumed by all nodes. If the **Truncate Before Sync** property is set to **true**, the population will operate in **SNAPSHOT** mode.



Note:

The job wakes up every BG_REF_SYNC_INTERVAL_SEC minutes, so the defined Time Interval may shift by a few minutes.

Built-in tables in Common Schema

- **_k2_read_pos**

Stores the consumer's offset in Kafka and the timestamp of the latest update (updated only after a message has been fully processed).

This information is used for backup functionality, as detailed later in the document.

```
fabric>select * from ref_crm._k2_read_pos;
|table_name|position|update_time|
+-----+-----+-----+
|OFFER     |4567    |1729595783093|
|PAYMENT   |4544    |1729595784109|
|BALANCE   |4134    |1729596030233|
(3 rows)
```

- **_k2_objects_info**

See Course 1 for details.

common_local_trx

If your implementation requires the transaction to be applied immediately - for instance, if your logic needs to use the transaction data right after applying it within the same session—use the command: **set common_local_trx=true;**

- **When common_local_trx is set to true:**

- The session will open a transaction on the Common DB file and apply the DML commands immediately.
- Once commit is executed, the transaction will be rolled back locally and sent via Kafka to be consumed by all Fabric nodes, including the local one.

- **When common_local_trx is set to false:**

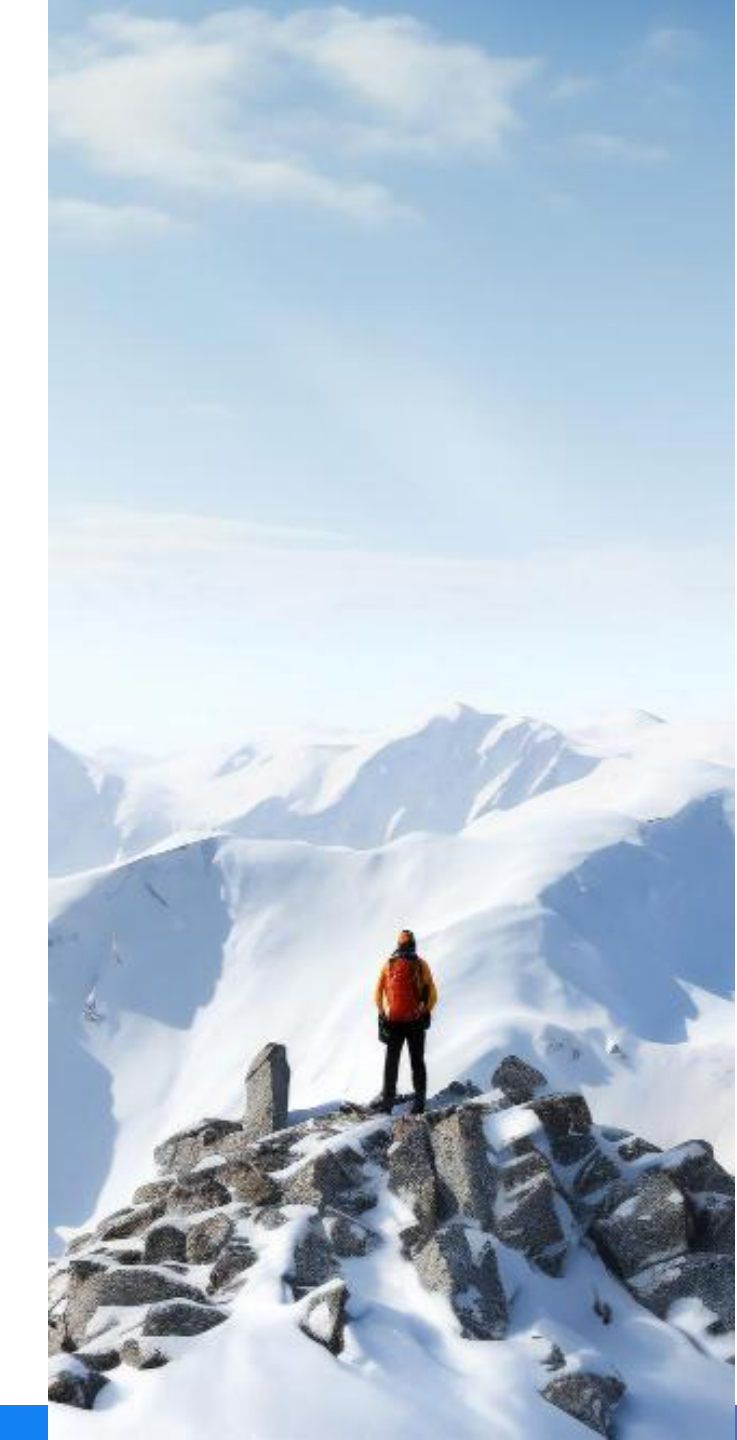
The updated data cannot be viewed until commit is performed, the transaction is sent via Kafka and the local Fabric node consume the commands and updates its local Common DB.



Note:

When common_local_trx is set to true, the transaction locks the SQLite file until a rollback or commit is executed. During this time, the Kafka consumer for this table cannot proceed and remains on hold until the transaction is completed.

Therefore, ensure that the transaction is closed as soon as possible.



Fabric commands for Common table using population

REF_SYNC

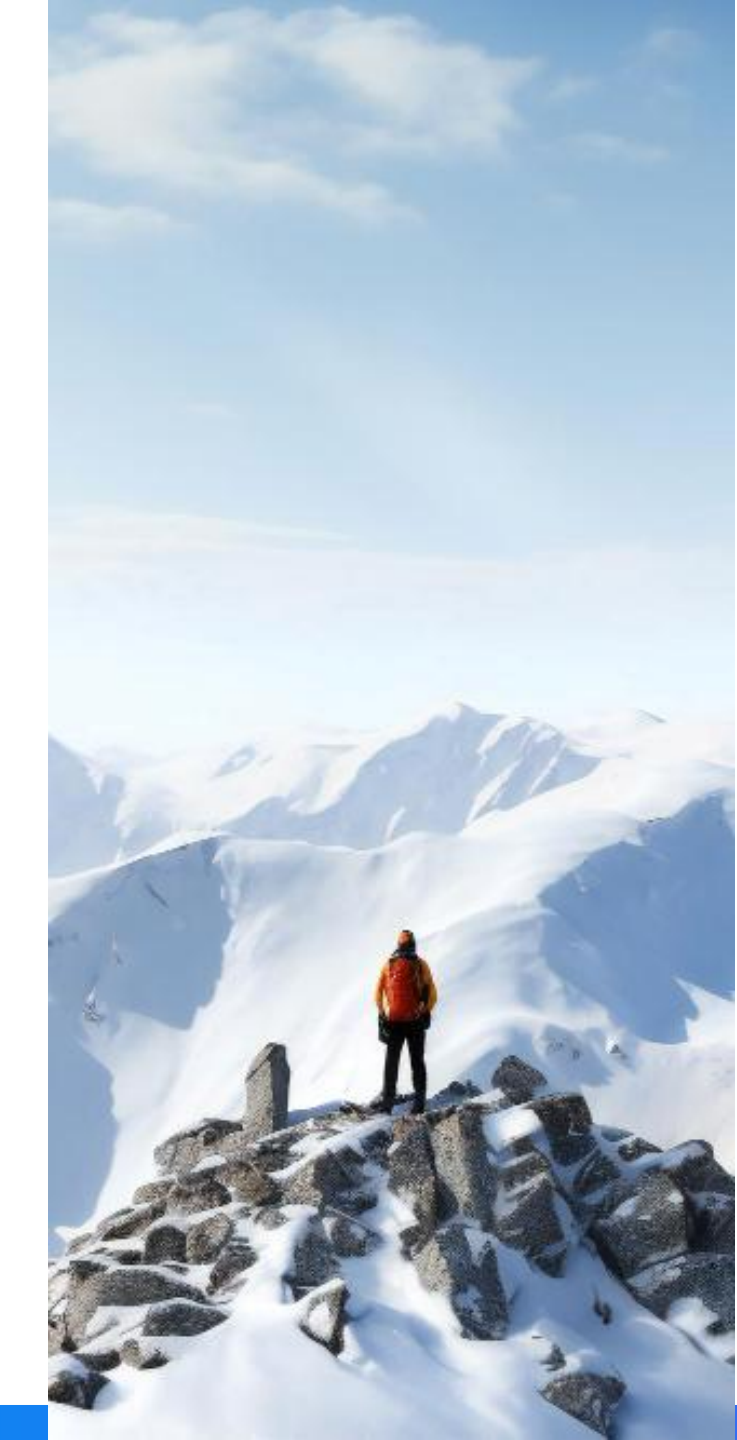
Start sync job for the specified common table(s).

```
ref_sync [LU_NAME='lu name'] [TABLES='ALL' or '<table 1,table 2,etc...>']  
[FORCE=true/false];
```

- LU_NAME: Syncs all tables listed in the LUT Reference list property.
- FORCE: Determines whether to force data sync from the source (default: true).

The command sends a request to the COMMONAREA_TABLE_SYNC by updating job parameters in k2_jobs. Once a Fabric node picks up the job, the population process begins.

If ref_sync is executed while another ref_sync is already running, an error message will be displayed.



Fabric commands for Common table using population

REF_CANCEL

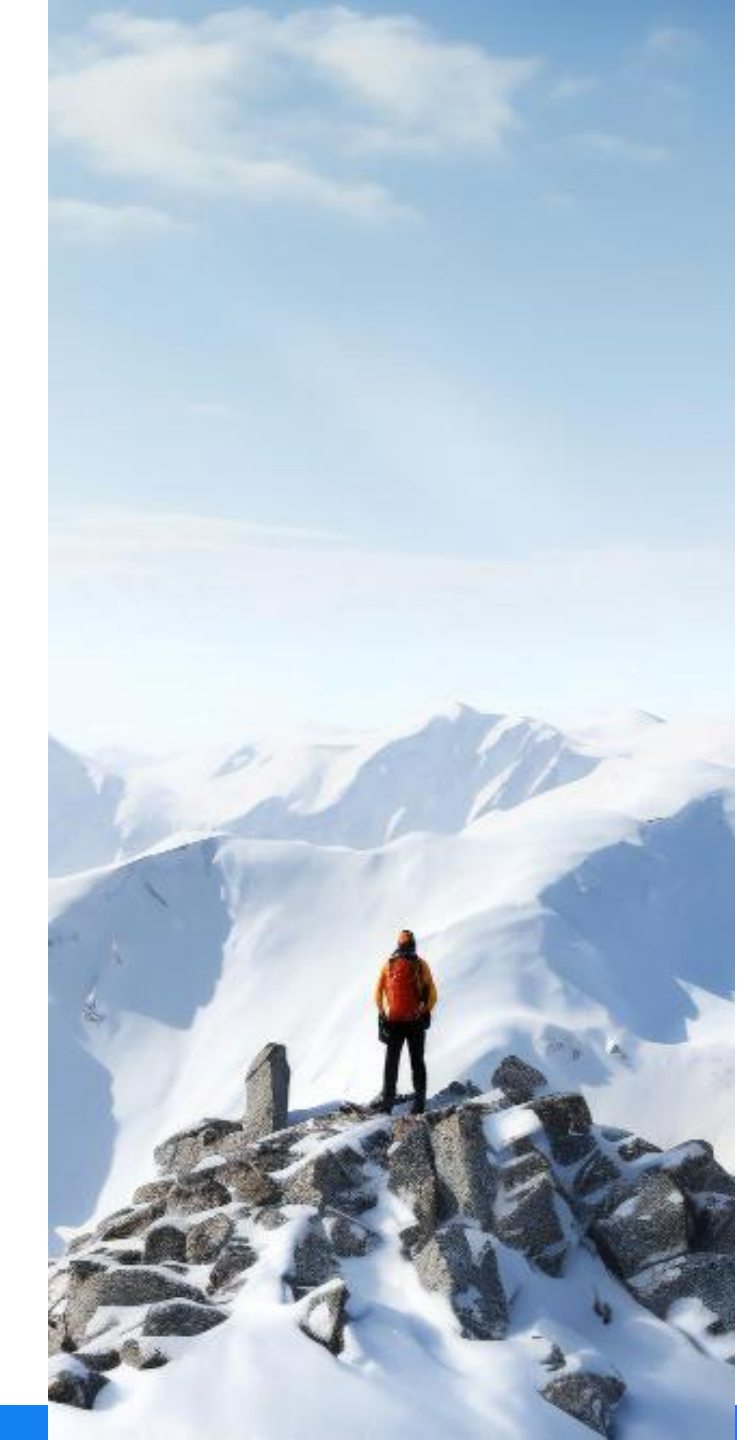
Cancel running ref_sync activity.

```
ref_cancel TABLE_NAME=table1 ;
```

- If the job hasn't started yet:
The ref_sync settings will be rolled back to cancel immediate execution (handled in k2_jobs).
- If population already completed:
 - Using ref_status - check current offset of each node.
 - If any of the nodes reached the commit – ref_cancel cannot cancel the ref_sync.
 - Else – starting from the largest current offset found in the ref_status, find the first and last offsets of LONG transactions, waiting in Kafka.
 - *If one of the nodes already working on the population message, it'll stop in the middle, rollback and skip
 - Send by jdbc to all the nodes, to skip all messages in this range.



Note: When using ref_cancel command, the population will not re-try automatically.



Fabric commands for Common table using population

REF_STATUS

Provides tables sync status for the specified reference tables across all nodes.

In addition, it provides the 'Current Session Transaction', i.e. the status of the latest transaction, executed on the table in the current session.

REF_STATUS [LU_NAME='lu name'] [TABLES='ALL' or '<table 1,table 2,etc...>']
[SCOPE='table' or 'population'];

REF_STATUS Results (SCOPE = 'table')

1. **Table Name:** The name of the reference table.
2. **Node:** The ID of the node.

Fabric commands for Common table using population

REF_STATUS

3. Status:

- **WAITING_FOR_SYNC:**

- *The table is waiting to be synced for the first time - OR -*
- *A REF_SYNC command was issued, but the job hasn't started yet.*

Looking at k2_objects_info: start_sync_time < request_time.

Note: If the table is waiting to be synced for the first time, GET operations using this table will wait for synchronization.

- **IN_SYNC:**

The table is in the process of being synced:

- *Syncing for the first time.*
- *Sync triggered manually via REF_SYNC in the current session (all other sessions see **IN_BACKGROUND_SYNC**).*

Looking at k2_objects_info: If (start_sync_time > end_sync_time) and (start_sync_time > request_time).

- **IN_BACKGROUND_SYNC:**

The sync is running in the background, triggered by:

- *Another session invoking REF_SYNC.*
- *The COMMONAREA_TABLE_SYNC job is running*

- **READY:** The table is fully synchronized.

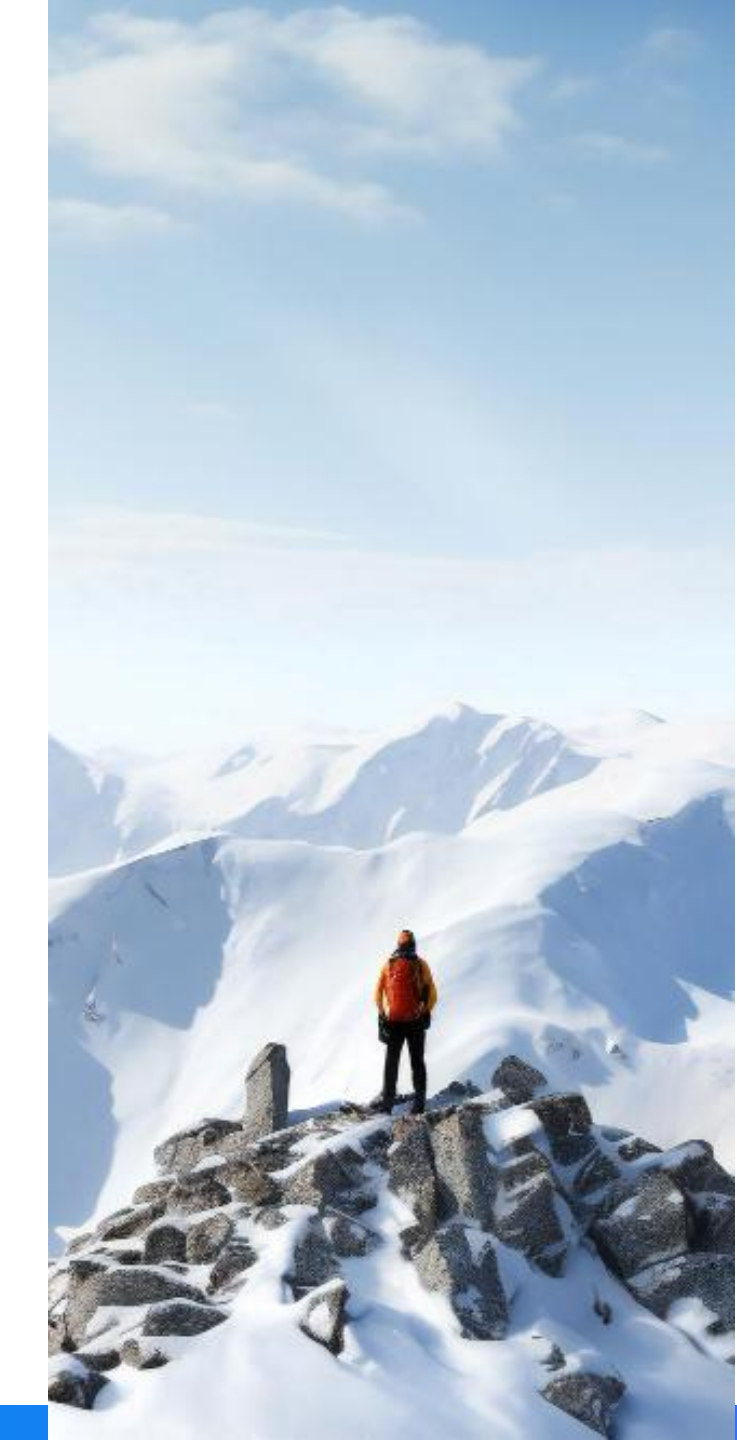
- **READY_WITH_BACKLOG:** The table is fully synchronized. There is a Kafka/queue backlog (messages that are not related to the population trx).

Note: If REF_SYNC was triggered in the current session, GET operations can proceed without waiting.

Fabric commands for Common table using population

REF_STATUS

4. **Backlog:** Number of pending messages in Kafka (main topic).
5. **Offset:** Current Kafka offset (taken directly from Kafka, not from _k2_read_pos).
6. **Offset Duration (min):** How long (in minutes) the current offset has been processing. Resets to 0 when the transaction completes.
7. **Num of Messages:** For long transactions, the number of commands processed in the current transaction.
8. **Transaction ID:** The ID of the currently processed transaction or snapshot.
9. **Transaction Type:** LONG_SNAPSHOT, SHORT_SNAPSHOT, SHORT_TRANSACTION, LONG_TRANSACTION, IDLE (no activity since the last restart).
10. **Sub Status:** Status of the write operation to the SQLite file.
Options: In process, Index rebuild (for snapshot messages), Done, or Failed.
11. **Current session transaction** – population status (writing to Kafka)
 - **Started:** Transaction started but not committed.
 - **Commit in progress:** Commit is ongoing.
 - **Completed:** Commit successfully completed.
 - **Rollback in progress:** Rollback is ongoing.
 - **Sync Error:** An error occurred during sync.
12. **Notes:** Not in use.



Fabric commands for Common table using population

REF_STATUS

REF_STATUS Results (SCOPE = population')

Shows k2_object_info from all the nodes

1. table_name: the name of the reference table
2. population: the name of the population querying the external sources
3. verified time: timestamp when last sync was verified
4. start/end sync time: Sync time start and end times
5. start/last write time: Write time of first and last message
6. next planned sync: Timestamp for next sync
7. sync error: error message
8. node: The ID of the node
9. notes: details – not in use

A vertical photograph on the left side of the slide shows a person in a red jacket and dark pants standing on a rocky, snow-covered mountain peak. The person is looking out over a vast, snow-covered mountain range under a clear blue sky with some light clouds.

Fabric commands for Common table using population

REF_SYNC_WAIT

Used by a session to wait for the completion of a REF_SYNC command or an insert, delete, or update operation on a Common table within the same session.

`REF_SYNC_WAIT [LU_NAME='lu name'] [TABLES='ALL' or '<table 1,table 2,etc...>'];`

This command pauses the session until the sync process finishes, respecting the **Sync Wait Timeout** property configured in the Studio for the table. The command should be executed after a REF_SYNC command or any insert, delete, or update operation on a Common table.

CommonDB Backup

When adding a new node to Fabric cluster, or for any issue with Common Area on one of the fabric clusters, a backup file can be used to create the local Common Area DBs

REF_BACKUP

Use the following command to back up the Common Area schemas:

```
REF_BACKUP [SCHEMAS='ALL' or '[schema 1, schema 2, etc...]'];
```

The backup files are stored in the location specified by the DEFAULT_GLOBAL_STORAGE_TYPE parameter:

```
## LU Metadata files storage (project artifacts zip jar): SYSTEM_DB/S3/AZURE_BLOB_STORE/GCS/NFS (Default: SYSTEM_DB)  
#DEFAULT_GLOBAL_STORAGE_TYPE=SYSTEM_DB
```

To store backups in a different location, add a new entry in config.ini:
[fabricdb].COMMONS_BACKUP_DEFAULT_STORAGE

**Note:**

The backup files must be stored in a shared location accessible to all nodes

CommonDB Backup

```
fabric>ref_backup;  
|Schema |Status |  
+-----+-----+  
|ref_crm|Success|  
|ref_sor|Success|  
  
(2 rows)
```

```
1 select *  
2 | from k2_commons_shanitestapli_alpinist.entity  
3
```

Columns to show ▼ ↓ CSV

Displaying Records: 2 | Query executed successfully | Duration: 11 ms | Affected Records: 0

< id >▼	< batch_id >▼	< chunks_count >▼	< data >▼	< key_desc_id >▼	< schema_hash >▼	< sync_version >▼
ref_crm		1	{"hb":["76,90,52,66,108,111,99,107,3...	0	-1483934405	1729352024593
ref_sor		1	{"hb":["76,90,52,66,108,111,99,107,3...	0	-1899278525	1729352024593

Backup Process

- During the backup, the database file undergoes **vacuum and maintenance** to optimize storage.
- Backup files also retain the **last offset** consumed by the Kafka consumer. This ensures that any consumer using the backup can resume from the correct offset in Kafka.



CommonDB Backup

REF_BACKUP_DELETE

Deletes a backup of the specified Common schema(s) from the configured storage.

ref_backup_delete [SCHEMAS='ALL' or '<schema 1,schema 2,etc...>']

```
fabric>ref_backup_delete schemas='ref_sor';
|Schema |Status |
+-----+-----+
|ref_sor|Deleted|

(1 row)
```

1 select *
2 from k2_commons_shanitestapli_alpinist.entity
3

Columns to show

Download CSV

Displaying Records: 1 | Query executed successfully | Duration: 20 ms | Affected Records: 0

< id ▾ ▴	< batch_id ▾ ▴	< chunks_count ▾ ▴	< data ▾ ▴	< key_desc_id ▾ ▴	< schema_hash ▾ ▴	< sync_version ▾ ▴
ref_cm		1	["hb":["76.90.52.66,108,111.99,107,3...	0	-1483934405	1729362497524

CommonDB Backup

REF_BACKUP_DOWNLOAD

Downloads a snapshot of the specified Common schema(s) from the configured storage to the specified destination path.

```
ref_backup_download [SCHEMAS='ALL' or '<schema 1,schema 2,etc...>'] DESTINATION='path name';
```



CommonDB Backup

Using Backup Files to Recover a Corrupted Common Area or Add a New Node to the Cluster

When a node starts (whether an existing node or a new one added to the cluster), Fabric performs the following steps automatically:

1. Initialize consumers and producers for each table
2. Check for local Common files:
 - If a local Common file exists - Begin reading from the offset stored in k2_read_pos for each table.
 - If no local Common file exists - Check if a backup is available.
If available - download it to the local node and begin reading from the last offset in the backup.
If the offset in the backup is no longer available in Kafka, the backup is invalid and cannot be used.
3. If no valid backup exists - request a full snapshot from another node:
 - A COMMONAREA_TABLE_REPLICATE job is created.
 - One of the nodes will pick up the job. If this node does not have a 'valid' Common table (i.e., its current offset is not available in Kafka), it will exit the job, allowing another node to take ownership. Otherwise, it will begin sending the snapshot to Kafka.
 - The snapshot is sent to Kafka with an **optional transaction flag**, and any node with an invalid table can consume it.

CommonDB Backup

Backup Practices and Recommendations

1. Manage Kafka and Snapshot Retention:

Ensure that the retention period for Kafka topics and Cassandra snapshots is longer than the backup interval; otherwise, the backup may become invalid since not all messages in Kafka, since the time of the backup, will still be there.

- Cassandra Configuration:

Set the retention period using: `[common_area_system_db_config].COMMONS_TABLE_TTL`

Default: 604800 seconds (7 days)

- Kafka Configuration:

Adjust the retention settings in the Kafka `server.properties` file.

2. Daily Backups:

- Schedule a job to back up all Common tables daily.

Note: when setting up the job interval to a very long period, the restore process can take longer to run (because more messages will need to be consumed from Kafka).

- In your implementation, add a BroadwayJob Actor to the `deploy.flow` Broadway flow in the References LU that will run the `REF_BACKUP` command for all the relevant schemas. It will start the common tables backup job when deploying the References LU.

CommonDB Backup

Backup Practices and Recommendations

3. Before Adding a New Node:

- Create a backup of all Common schemas.

4. Fixing a Corrupted Common DB on an Existing Node (Fabric 7.1+):

- Create a backup from a node with a valid Common DB.
- On the corrupted node:
 1. *Stop Fabric.*
 2. *Delete the local Common file.*
 3. *Restart Fabric.*

CommonDB Backup

Fixing corrupted Common Area in Fabric versions prior to 7.1:

1. For Fabric versions prior to 7.0:
 1. Sync one fabric node and stop it
 2. Stop relevant nodes
 3. Go to storage/common on the synced node, and make sure the .wal file doesn't exist or empty.
If not – run vacuum manually on sqlite3
 4. Go to the relevant node's storage\common and delete all files (3 at most)
 5. Copy common.db from the synced node to relevant nodes
 6. On relevant nodes, for Kafka consumer group named as node id - move offset to end.
 7. Restart nodes.
2. For Fabric version 7.0:
 1. Sync one fabric node and stop it
 2. Stop relevant nodes
 3. Go to storage/common on the synced node, and make sure the .wal file doesn't exist or empty.
If not – run vacuum manually on sqlite3
 4. Go to the relevant node(s) storage\common and delete all files (depending on the number of schemas that you defined in Common DB)
 5. Copy all the schemas of Common DB from the synced node to relevant nodes
 6. Restart nodes.

Vacuum & maintenance

Common tables operate in **WAL (Write-Ahead Logging) mode**, with Fabric periodically running vacuum and maintenance tasks between transactions.

If Fabric cannot perform maintenance due to high write activity, the WAL file may grow excessively.

For a short-term solution, follow the steps below.

For a long-term solution, investigate what prevents Fabric from running regular maintenance. Possible causes include:

- High volume of updates on the schema – consider splitting the schema into multiple schemas.
- Processes (jobs/flows) locking the file for extended periods.

Short-Term Solution for Large WAL Files:

1. Ensure no Snapshot transaction is running.
2. Stop Fabric – Fabric will attempt to run maintenance during shutdown.
3. If the WAL file remains large, manually run vacuum using sqlite3.
4. Restart Fabric.

Delete Common Table

A Common table can be deleted through Fabric Studio by removing it and deploying the changes.

If the DELETE_TOPICS_ON_DROP=true setting is enabled in config.ini, the Kafka topic associated with the dropped table will also be deleted.

Note: Avoid dropping the Kafka topic if another client is using it.

* **In the cloud environment**, when a space is destroyed, Fabric does not delete resources that were not created by it. For example, this includes topics, Cassandra tables, and other resources not managed by the cloud manager.

Common Tables Configuration

The [default_pubsub] section is used by Common DB processes for connecting to Kafka. To customize Kafka settings for Common DB, use the [common_area_pubsub] section, which can override specific settings without redefining all parameters.

[default_pubsub]

- **TYPE**=KAFKA - pubsub type can be one of [MEMORY, KAFKA, NO_OP, ERROR]
 - MEMORY - execute the message handling via an internal queue that runs on localhost. This type can only be used for debug purpose.
 - NO_OP - do not send or receive the messages. This type can only be used for debug purpose.
 - ERROR - simulate throwing an Unsupported Operation exception. This type can only be used for debug purpose.
- **POLL_TIMEOUT**=-1 - The timeout to wait for a new message. If the timeout elapses the collection will come to an end. If set to -1 the wait will be forever.
- **MAX_POLL_RECORDS** – Max number of records that the consumer reads from Kafka in each poll (default: 500). For performance optimization.

Common Tables Configuration

- **TRANSACTION_MODE** - Determines how the publisher handles transactions. Options include **Async**, **Broker**, and **Ignore**:

Async Mode

- Description:
 - *Messages are aggregated in an internal queue and sent asynchronously to Kafka upon commit (or discarded on rollback).*
- Key Points:
 - **No Guaranteed Delivery:** *If any message fails, the remaining messages are still sent.*
 - *The commit command raises an exception for failed messages. Check the logs for details.*
 - *Faster than **Broker** or **Ignore** modes.*
 - *Recommended for scenarios where transactions are not critical.*
- Queue Size:
 - *Controlled by PUB_QUEUE_LIMIT (a hidden parameter in config.ini).*

Common Tables Configuration

Broker Mode

- Description:
 - *Fully supports transactions, sending messages directly to Kafka without using an internal queue.*
- Key Points:
 - *On `commit`, the transaction is committed to Kafka.*
 - *On `rollback`, the transaction is rolled back in Kafka as well.*
 - *Transaction size is limited only by Kafka, with no additional restrictions from Fabric.*
 - *Suitable for scenarios requiring transaction guarantees.*

Ignore Mode

- Description:
 - *Messages are sent synchronously, one by one, and committed immediately.*
- Key Points:
 - *No need to execute `commit` in Fabric.*
 - *Transactions are not supported, and each message is handled individually.*
 - *Recommended for scenarios where transaction handling is unnecessary.*

Common Tables Configuration

[common_area_config]

- Error handling:
 - **CONSUMER_IDLE_TIME**=60000 – Maximum timeout (in milliseconds) to wait for the next snapshot in Kafka. If exceeded, an IDLE_TIMEOUT error will be thrown.
 - **CASSANDRA_WAIT_MESSAGE_TIMEOUT**=60000 - Maximum timeout (in milliseconds) to wait for the next snapshot in Cassandra. If exceeded, an IDLE_TIMEOUT error will be thrown.
 - **OPERATION_RETRIES_COUNT**=-1 - Maximum retries for common SQLite errors (e.g., SQLITE_BUSY, SQLITE_LOCKED). -1 indicates unlimited retries.
 - **OPERATION_RETRIES_WAIT_MS**=1000 – Interval (in milliseconds) between each retry attempt defined by OPERATION_RETRIES_COUNT.
- **SYNC_JOBS_AFFINITY** - Affinity for common sync jobs (comma-separated). Default: no affinity.
- **DELETE_TOPICS_ON_DROP** - If set to true, removes all unused topics upon a drop operation.
- **TRANSACTION_BULK_SIZE**=100 – transaction bulk size
- **MAX_TRANSACTIONS_COMMIT**=100 – Specifies when to commit to SQLite during snapshot message processing

Common Tables Configuration

[common_area_config]

- **MESSAGES_INTERNAL_BROKER_TYPE**=PUB_SUB - Defines the storage for long transaction distribution (SYSTEM_DB or PUB_SUB).
- **MIN_TRANSACTION_LONG** For Cassandra: Ensures efficient transaction handling by creating snapshots in Cassandra for smaller tables instead of sending them over Kafka.
- INTERNAL_QUEUE_SIZE=500 – **not in use**
- PROCESS_UPDATE_MESSAGE_RETRIES_COUNT=3 – **not in use**

[common_area_system_db_config]

- **COMMONS_SNAP_TABLE**=snapshots – Specifies the name of the Cassandra table used for snapshots.
- **COMMONS_TABLE_TTL**=604800 - Defines the TTL (time-to-live) for the snapshot table in seconds. Default is one week
- **COMMONS_KEYSPACE**= Specifies the Cassandra keyspace for the snapshots table.

Performance Considerations

1. Common Table Population

When possible, use IIDFinder to update your table instead of resyncing the entire dataset for each update.

For Full Resyncs:

- Set Truncate Before Sync to True:
This ensures the table is cleared before the new data is loaded.
- Optimize Sync Policy:
 - *Avoid short sync intervals to prevent continuous sync cycles.*
 - *For example, if the source table updates once daily, set the sync interval to match.*

2. Fetching Data from Common Tables

- Use ludb.fetch or fabric.fetch for Queries:
Avoid using FabricDB.fetch (the local Fabric interface) as it can negatively impact performance due to overhead from TCP communication.
- Optimize Index Usage: create indexes only when necessary to avoid unnecessary system complexity.

3. Transaction Management: prefer Kafka for Long Transactions

Performance Considerations

- Adjust bulk settings:
 - Adjust **TRANSACTION_BULK_SIZE** value based on project needs. For large tables with high write activity, increase this parameter.
Note: Kafka has a limitation on the maximum message length.
 - Adjust **MIN_TRANSACTION_LONG** when working with Cassandra. Define this parameter for smaller tables to ensure efficient transaction handling.
- Adjusting **MAX_TRANSACTIONS_COMMIT**: Specifies when to commit to SQLite during snapshot message processing
Tradeoff: A higher value improves performance but increases the risk of failures and retries for the entire transaction.
- Schema Splitting: Distribute tables across schemas to reduce locking and improve performance, as explained earlier.
- If the GET process updates a Common table, ensure updates occur only when there are actual changes to avoid unnecessary writes.

Best Practices

- **Common Table Population**

- Use IIDFinder for Updates instead of resyncing the entire dataset for each update.
- Use Upsert Mode if re-processing transactions may be required.
- For Full Resyncs, set Truncate Before Sync to True, to enable Snapshots management
- Optimize Sync Policy: Avoid short sync intervals to prevent excessive sync cycles. For example: If the source table updates once daily, set the sync interval accordingly.

- **Use FTS for Large Text Queries:**

For querying large text collections (e.g., articles, reports, chats), consider using SQLite's built-in Full-Text Search (FTS) feature.

Trade-offs: FTS may reduce insert and update performance and increase storage requirements.

Optimization Tip: split the table into two:

- FTS Table: Contains primary key fields and columns needed for search.
- Secondary Table: A regular table with primary key fields and other columns.

Best Practices

- **Transaction Management**

- Prefer Kafka for Long Transactions. Kafka is better suited for long transactions than Cassandra.
- Ensure Message Length is Within Kafka Limits. Avoid sending overly long messages that exceed Kafka's size limitations.
- Schedule Long Sync Transactions Strategically. Execute long sync transactions during off-peak hours (low API volumes) to minimize server performance impact.

Design Considerations

Category	ES	REF	Postgres	Cass
Real-time	X (CDC)	X	V	V
High transaction volumes	V	X	V	V
FTS with Large Volumes	V	X	X	X
FTS with Small to Medium Volumes	V	V	X	X
Fuzzy search	V	X	X	X

*Note: The above table does not address the complexity of each application, including aspects like implementation, installation, maintenance, and cost.

Common Tables - JMX Stats

Under the '**transactions**' section, the following statistics are available for monitoring Common table operations:

- **commonUpdate**: Tracks the number of commands (insert, update, delete) executed for each table.
- **commonTransactions**: Tracks the number of transactions executed for each table.
- **commonSnapshot**: Tracks the number of snapshots executed for each table.
- **commonSnapshotDuration**: Measures the duration of full snapshot processing for each table.
- **commonMessagesBulkProcessDuration**: Measures the processing time for messages that exceed the bulk size.